

BIG DATA

COMPREHENSIVE EXAM REVISION NOTES

COMPSCI4064 / COMPSCI5088

University of Glasgow

Based on 8 Years of Past Papers (2015-2025)

Topics Ranked by Exam Frequency & Recency

Exam Format: 60 marks | 1.5 hours | 3 compulsory questions | Closed book

Table of Contents

Table of Contents.....	2
1. MapReduce (14 appearances, 146 marks)	5
1.1 Core Concept	5
The Two Operations	5
Word Count Example (the canonical exam example)	5
The Shuffle and Sort Phase (asked 2018 for 14 marks)	5
Key Classes Beyond Map/Reduce/Main (asked 2025).....	6
Designing MapReduce Jobs (exam technique)	6
Example: News Summarization (2023 exam, 9 marks)	6
Example: In-link Counting in Directed Graph (2018, 18 marks)	6
2. Distributed Storage & HDFS (12 appearances, 94 marks)	8
2.1 HDFS Architecture.....	8
Why 128MB Block Size (asked 2019 for 8 marks).....	8
HDFS Read Process (asked 2018 for 10 marks).....	8
HDFS Write Process (Pipelining)	9
Default Replica Placement Policy (asked 2019 for 10 marks)	9
NameNode Recovery After Reboot (asked 2018 for 10 marks).....	9
NFS vs HDFS Comparison (asked 2023).....	9
Block Replica States	10
Replication Factor Effects (asked 2023 Practice Exam)	10
3. BigTable / HBase (12 appearances, 67 marks)	11
3.1 What is BigTable/HBase?	11
Six Key Properties	11
Data Model (critical for exam)	11
Unique Cell Identifier (asked 2019)	11
On-Disk Storage Format (asked 2018 for 5 marks).....	11
HBase Architecture	12
Data Lookup Procedure (asked 2016 for 10 marks)	12
HBase vs HDFS.....	12
4. Apache Spark (8 appearances, 53 marks)	13
4.1 Core Concepts.....	13
RDDs (Resilient Distributed Datasets)	13
Transformations vs Actions (critical distinction)	13
Lazy Evaluation and DAGs (asked 2019, 2020).....	13

Narrow vs Wide Dependencies (asked 2020 for 9 marks).....	13
Spark Architecture	14
Spark vs Hadoop (asked repeatedly)	14
Why Stateful Processing is Costly in Spark (asked 2025)	14
5. Cassandra (8 appearances, 42 marks)	15
5.1 What is Cassandra?	15
Architecture.....	15
Partitioning.....	15
Replication and Arcs	15
Virtual Nodes (asked 2023).....	15
Consistency Levels (asked 2018, 2019, 2020).....	16
Quorum Formula (asked repeatedly).....	16
Cassandra vs HBase	16
6. Hadoop (8 appearances, 49 marks)	17
6.1 Hadoop Architecture (v1).....	17
23-Step Job Execution Flow	17
Overhead Costs (asked 2023)	17
Data Locality Principle.....	17
YARN (Yet Another Resource Negotiator)	17
YARN Containers vs Modern Containers (asked 2023 M-level)	18
Hadoop Versions	18
7. Big Data Fundamentals (7 appearances, 39 marks).....	19
7.1 The 6 Dimensions (V's)	19
Vertical vs Horizontal Scaling.....	19
CAP Theorem (asked 2019).....	19
BASE vs ACID	20
Batch vs Stream Processing	20
8. Data Structures: LSM Trees & Bloom Filters (4 appearances, 28 marks).....	21
8.1 LSM Trees (Log-Structured Merge Trees)	21
Components	21
Write Process	21
Read Process	21
Trade-offs (asked 2018, 2020).....	21
8.2 Bloom Filters	22
How They Work	22
Key Formulas.....	22

9. Cluster Management & Containers (2 appearances, 13 marks)	23
9.1 Software Sandboxes	23
9.2 Kubernetes	23
Components	23
Building Blocks	23
10. Legal & Ethical Considerations (1 appearance, 6 marks)	24
Four Core Ethical Concepts	24
Key Laws	24
11. Batch vs Streaming (3 appearances, 12 marks).....	25
Why Streaming is Challenging	25
Back Pressure	25
Why Hadoop is Bad for Streaming (asked 2025).....	25
12. ZooKeeper & DHT / Overlay Networks	26
12.1 ZooKeeper (1 appearance, 3 marks)	26
12.2 DHT / Overlay Networks (2 appearances, 14 marks).....	26
Consistent Hashing	26
Finger Tables	26
Order-Preserving Hashing	26
13. Exam Strategy & Top Expected Questions.....	27
13.1 Exam Format (2026).....	27
13.2 Answer Technique	27
13.3 Top 10 Most Expected Questions for 2026	27
13.4 Key Formulas to Memorize	27

1. MapReduce (14 appearances, 146 marks)

EXAM PATTERN: Appeared in 7 of 8 years (2016-2025). Highest weighted topic.

Common formats: Design a MapReduce job for a scenario (9-18 marks), explain the shuffle phase (14 marks), write pseudocode for Map/Reduce functions, explain MapReduce vs Spark.

2025: Types of classes needed beyond Map/Reduce/Main (6 marks)

2023: Design MapReduce for news summarization pipeline (9 marks)

2020: Design MapReduce for SELECT DISTINCT + combiner extension (8 marks)

2019: Optimal number of reducers, disadvantage of reduce-side join (14 marks)

2018: Design MapReduce for counting in-links in directed graph + top-k variant (18 marks), shuffle phase explanation (14 marks)

2017: Write MapReduce Java code for inverted index and k-NN (40 marks)

1.1 Core Concept

MapReduce is a programming model for processing large datasets in parallel across a cluster. Published by Google in 2004, it became the foundation of Big Data processing.

The Two Operations

Map(Key1, Value1) → List[<Key2, Value2>] — Transforms input records into intermediate key-value pairs. Runs in parallel across data partitions. Used for extraction, filtering, and transformation tasks.

Reduce(Key2, List[Value2]) → List<Value2> — Aggregates all values associated with the same key. Used for aggregation, counting, and merging tasks.

Word Count Example (the canonical exam example)

Map: Input <docID, document> → Output List<word, 1> for each word in document.

Reduce: Input <word, [1, 1, 1, ...]> → Output <word, total_count>.

The Shuffle and Sort Phase (asked 2018 for 14 marks)

Between Map and Reduce, data undergoes a critical reorganization:

1. Map output stored in an in-memory circular buffer (default 100MB).
2. When buffer reaches threshold (80%), contents are sorted by key, partitioned by reducer, and spilled to disk.
3. Combiners may run on spilled data (local pre-aggregation to reduce network transfer).
4. After all map input is processed, spill files are merge-sorted into a single sorted, partitioned file.

5. Mapper notifies JobTracker via heartbeat that it has completed.
6. Each reducer contacts completed mappers and transfers its relevant partition (the shuffle).
7. Reducer performs a recursive merge-sort on incoming data (mix of in-memory and on-disk).
8. Merged, sorted data is fed to the reduce() function.

Key Classes Beyond Map/Reduce/Main (asked 2025)

InputFormat: Converts raw HDFS blocks into usable key-value pairs. Types include TextInputFormat (line-by-line), KeyValueTextInputFormat, SequenceFileInputFormat, NLineInputFormat, DBInputFormat.

OutputFormat: Controls how reduce output is written. Types include TextOutputFormat, SequenceFileOutputFormat, MapFileOutputFormat, NullOutputFormat.

Partitioner: Determines which reducer receives which keys. Default is hash-based. Custom partitioners enable load balancing.

Combiner: Optional local reducer that runs after map phase to reduce data transfer. Must be commutative and associative.

Designing MapReduce Jobs (exam technique)

HOW TO ANSWER: MapReduce Design Questions

1. Identify what is a per-record operation (Map) vs what needs grouping (Reduce).
2. Define your key: what should records be grouped by?
3. State input format: <key_type, value_type> for both Map and Reduce.
4. State output format for both Map and Reduce.
5. Justify WHY each operation is a Map or Reduce (link to scenario).
6. Consider if Combiners can be used (only if operation is commutative + associative).
7. For chained jobs: explain why multiple Map-Reduce rounds are needed.

Example: News Summarization (2023 exam, 9 marks)

Scenario: Extract content from articles, identify topics, group paragraphs by topic, summarize.

Map 1: Input <articleID, article> → Content extraction + topic identification → Output <topic, paragraph>. Justification: extraction and topic ID are per-article transformations.

Reduce 1: Input <topic, List<paragraph>> → Summarization → Output <topic, summary>. Justification: summarization requires all paragraphs for a topic, which is an aggregation.

Example: In-link Counting in Directed Graph (2018, 18 marks)

Basic: Mapper reads edge (source, target), emits <target, 1>. Reducer sums counts. Combiner does local summing.

Top-k variant: Same mappers/combiners. Reducers maintain a sorted list in memory (initialized in `setup()`), insert computed pairs, trim beyond position k. Emit final list in `cleanup()`. Driver merges k-sized outputs from each reducer.

2. Distributed Storage & HDFS (12 appearances, 94 marks)

EXAM PATTERN: Appeared in 7 of 8 years (2015-2023). Second highest weight.

Common formats: HDFS read/write process steps, block size justification, replica placement trade-offs, NameNode recovery, NFS vs HDFS comparison.

2023: NFS+RAID6 vs HDFS comparison (8 marks), distributed vs decentralized (3 marks), replication factor effects (15 marks)

2020: How HDFS client reads data (8 marks), NameNode vs DataNode hardware (6 marks)

2019: Why 128MB block size (8 marks), replica placement policy trade-offs (10 marks)

2018: HDFS read request steps with error handling (10 marks), NameNode recovery (10 marks)

2.1 HDFS Architecture

NameNode (Master): Stores all metadata in RAM — file-to-block mappings, block-to-DataNode locations, directory structure, permissions, modification times. Single point of failure (until Hadoop v3). Recovery files: Checkpoint (filesystem snapshot) + Journal (write-ahead log of all changes since checkpoint).

DataNode (Worker): Stores actual data blocks. Each HDFS block = 2 native files (contents + metadata with checksum/size). Reports to NameNode via heartbeats every 3 seconds, sending block reports with <block_id, timestamp, length>.

Why 128MB Block Size (asked 2019 for 8 marks)

Disk seek time is approximately 10ms. At a transfer rate of ~100MB/s, reading 128MB takes ~1.28 seconds. This makes seek time only ~1% of total transfer time, maximizing throughput. Smaller blocks would make seek overhead dominate. The large block size is designed for batch processing of large files, not random access of small files.

HDFS Read Process (asked 2018 for 10 marks)

1. Client contacts NameNode with file pathname.
2. NameNode returns block IDs and DataNode locations sorted by network distance (Rack Awareness).
3. Client contacts closest DataNode directly (NameNode not involved in data transfer).
4. DataNode streams block in 64KB packets, each with a checksum.
5. Client verifies checksum. If valid, data is available in client memory buffer.

Error handling: Checksum failure → report to NameNode, try next DataNode. Network failure → try next DataNode, mark faulty node. Client retries configurable number of times.

HDFS Write Process (Pipelining)

1. Client asks NameNode to choose DataNodes for replicas of first block.
2. DataNodes form a sequential pipeline (DN1 → DN2 → DN3).
3. Client sends data through pipeline as 64KB packets. Each node forwards to next while receiving.
4. Multiple packets can be in-flight simultaneously (pipelining).
5. Acknowledgements returned per packet. hFlush() periodically writes to journal for crash safety.
6. Each DataNode reports received blocks directly to NameNode.

Default Replica Placement Policy (asked 2019 for 10 marks)

Replica 1: Same node as writer (or random node in writer's rack). Replica 2: Random node in a DIFFERENT rack. Replica 3: Different node in the SAME remote rack as Replica 2.

Trade-off: Balances reliability (survives rack failure via cross-rack replica) with write bandwidth (only one cross-rack network transfer needed). Read bandwidth benefits from multiple locations.

NameNode Recovery After Reboot (asked 2018 for 10 marks)

1. NameNode reads checkpoint file and journal file from local disk.
2. Checkpoint is a complete filesystem state snapshot (directory structure, block-to-file mappings).
3. Journal transactions after the checkpoint timestamp are replayed, restoring all changes since last checkpoint.
4. NameNode waits for DataNodes to register via heartbeats.
5. Each DataNode sends block IDs and sizes in a Block Report.
6. NameNode stores block-to-DataNode mappings in memory. System is now operational.

NFS vs HDFS Comparison (asked 2023)

Feature	NFS	HDFS
Replication	None (relies on RAID)	Built-in (default 3 replicas)
Fault tolerance	Server = single point of failure	Survives node/rack failures
Small files	Efficient	Inefficient (NameNode overhead)
Large files	Limited by server	Optimized (parallel block reads)
Write model	Lock-based, single writer	Lease-based, write-once-read-many
Complexity	Simple to set up	Requires cluster management
Cost	Single server	Multiple commodity machines

Block Replica States

On DataNodes: RBW (Replica Being Written) → Finalized. If DN fails during write: RBW becomes RWR (Waiting Recovery) → RUR (Under Recovery) when DN comes back.

On NameNode: Under Construction → Under Recovery (if lease expires) → Committed (some replicas finalized) → Complete (all required replicas finalized).

Replication Factor Effects (asked 2023 Practice Exam)

Write throughput: Slightly reduced (pipeline adds $\sim N$ x network latency, negligible on fast networks).

Read latency: Potentially reduced (more replicas = closer replica available).

NameNode resources: More memory needed to track additional replica locations, more bandwidth for block reports.

DataNode resources: More disk usage across the cluster.

3. BigTable / HBase (12 appearances, 67 marks)

EXAM PATTERN: Appeared in 7 of 8 years (2015-2020, 2023). Heavy in earlier exams.

Common formats: HBase data model, on-disk storage format, data lookup procedure, HBase vs HDFS, region splitting, tablet size calculations.

2025: RegionServer storage mechanism (LSM tree) for fast access (7 marks)

2020: Row store vs column store (4 marks)

2019: What uniquely identifies a cell, what happens when tablet too large (6 marks)

2018: On-disk storage format of HBase row (5 marks)

2017: Calculate max storable data, how HMaster exclusivity ensured, unassigned tablet discovery (11 marks)

2016: HBase data lookup procedure via ZooKeeper (10 marks)

3.1 What is BigTable/HBase?

BigTable was developed by Google in 2005 for fast writes to complement GFS. HBase is the open-source implementation (2008), running on top of HDFS. It is a distributed, non-relational database focused on Consistency and Availability (CA in CAP).

Six Key Properties

Sparse (rows can have missing columns), Distributed (data blocks spread across machines), Persistent (always on persistent storage), Multidimensional (indexed by row key + column + timestamp), Sorted (lexicographically by RowKey), Map (maps RowKeys to Records).

Data Model (critical for exam)

Row: Group of cells sharing a RowKey.

Columns: Store data for each RowKey. One RowKey can have unlimited columns.

Column Families: Groups of columns stored together on disk. Each has its own HDFS directory (/hbase/table/CFx/).

Cell: Intersection of RowKey + Column Qualifier. Can have multiple versions via timestamps.

Regions: Contiguous sets of rows stored together. When a region exceeds max size, it SPLITS into two smaller regions (asked 2019).

Unique Cell Identifier (asked 2019)

A cell is uniquely identified by the tuple: {RowKey, Column Qualifier, Timestamp}. This is the composite key used in on-disk storage.

On-Disk Storage Format (asked 2018 for 5 marks)

Data is stored as key-value pairs. Key = composite of <RowKey, Column Family, Column Qualifier, Timestamp> plus size metadata. Value = column value + size. Entries with the same RowKey are stored contiguously, sorted by composite key. This means HBase is a COLUMN-ORIENTED store (not row store), because each Column Family is stored in its own directory.

HBase Architecture

HMaster: Assigns regions to RegionServers, creates/deletes tables, routes requests, handles compaction and failures.

RegionServer: Handles data requests. Each holds multiple regions. Contains 4 storage structures (LSM Tree): WAL (Write-Ahead Log), MemStore (recent writes in memory), BlockCache (recent reads), HFile (compressed spill files on disk = SSTables).

ZooKeeper: Communication bridge tracking state of HMaster and RegionServers. Triggers recovery on failure.

Data Lookup Procedure (asked 2016 for 10 marks)

1. Client queries ZooKeeper for the location of the META table.
2. Client reads the META table to find which RegionServer holds the target row's region.
3. Client contacts the RegionServer directly.
4. RegionServer checks MemStore first (fastest, most recent data).
5. If not in MemStore, checks BlockCache (recently read data).
6. If not cached, uses Bloom Filters to identify which HFiles might contain the data.
7. Reads relevant HFiles from HDFS and returns the result.

HBase vs HDFS

HBase is a column-oriented data store; HDFS is a file store. HBase supports random writes; HDFS does not. HBase performs incremental writes (buffered in memory). HBase reads smaller blocks (regions) than HDFS blocks. HBase operations are typically lower latency.

EXAM WARNING: Is HBase a replacement for file storage?

NO. HBase is a column store, not a file store. It cannot store arbitrary files like student documents. This was a trick question in 2023 Practice Exam (Q3b) — the marking guide notes it is deliberately checking if students understand the difference.

4. Apache Spark (8 appearances, 53 marks)

EXAM PATTERN: Rising topic — appeared in 4 of last 5 exams. Expect 6-10 marks.

Common formats: Spark advantages over Hadoop, lazy evaluation, narrow vs wide dependencies, RDD lineage, Spark code interpretation.

2025: Why stateful distributed training is costly in Spark (6 marks), streaming vs batch for ML (4 marks)

2023: Primary advantages of Spark over Hadoop (6-8 marks)

2020: Narrow vs wide dependencies, stage creation, performance implications (9 marks)

2019: Identify which line triggers RDD computation (4 marks), design Spark job (3 marks)

4.1 Core Concepts

Apache Spark (2009, UC Berkeley) was designed to solve Hadoop's limitations. Key insight: eliminate disk writes between stages by keeping data in memory. Founded on Resilient Distributed Datasets (RDDs).

RDDs (Resilient Distributed Datasets)

A collection of records that may or may not have a schema. Automatically partitioned when loaded. Partitions can be processed in parallel. RDDs are immutable — transformations create new RDDs.

Transformations vs Actions (critical distinction)

Transformations: Convert one RDD into another. Lazy — not executed until an Action is called. Examples: map, flatMap, filter, groupByKey, reduceByKey, join, union, intersection, repartition, coalesce, sortByKey, sample.

Actions: Trigger actual computation and return results to driver or write to storage. Examples: collect, count, reduce, first, take, takeSample, takeOrdered, saveAsTextFile.

Lazy Evaluation and DAGs (asked 2019, 2020)

Spark uses lazy evaluation — transformations are NOT scheduled until their output is actually needed by an Action. When an Action is called, Spark builds a Directed Acyclic Graph (DAG) of all required transformations, then reverse-engineers the minimal set of tasks needed. If an RDD is not required for the action, the transformations that produce it are never executed.

Narrow vs Wide Dependencies (asked 2020 for 9 marks)

Narrow dependency: Each partition of the parent RDD is used by at most one partition of the child RDD. Examples: map, filter, union. Can be computed in parallel without data shuffling. Multiple narrow transformations are pipelined within one stage.

Wide dependency: Each partition of the parent RDD may be used by multiple partitions of the child RDD. Examples: groupByKey, reduceByKey, join, repartition. Requires data shuffling across the network. Forces a stage boundary in the DAG.

Performance implication: Wide dependencies are expensive because they require network transfer (shuffle). The DAG scheduler groups consecutive narrow transformations into single stages for efficiency.

Spark Architecture

Driver: Program that executes the Spark job. Must remain alive until job completion. Contains the SparkContext.

Cluster Manager (Spark Master): Receives work, analyzes DAG, assigns work to workers. Contains DAG Scheduler and Task Scheduler.

Worker/Executor: JVM that executes tasks using ThreadPool. Each job gets its own Executor (no memory sharing between jobs).

Spark vs Hadoop (asked repeatedly)

Aspect	Hadoop MapReduce	Apache Spark
Processing	Disk-based (writes after every stage)	In-memory (up to 100x faster)
Job structure	Fixed Map → Reduce	Flexible DAG of transformations
Chaining	Expensive (disk I/O between stages)	Cheap (in-memory)
Languages	Primarily Java	Scala, Java, Python, R, SQL
Streaming	Not supported natively	Spark Streaming (micro-batches)
Caching	None	RDD caching in memory
Use cases	Batch processing	Batch + interactive + streaming

Why Stateful Processing is Costly in Spark (asked 2025)

Stateful applications require global state shared across tasks. In Spark this means: synchronization adds waiting time between stages, state storage can cause out-of-memory errors, network transfer of state between executors is expensive, and if an error occurs, state is lost requiring full batch re-execution (no incremental recovery).

5. Cassandra (8 appearances, 42 marks)

EXAM PATTERN: Appeared in 7 of 8 years. Focus on consistency levels and quorum.

Common formats: Consistency levels read/write behavior, quorum calculations ($R+W>N$), Cassandra vs HBase, virtual nodes, gossip protocol.

2025: Benefits of Cassandra over HBase for distributed crypto exchange (4 marks)

2023: Consistent hashing + order-preserving hashing in Cassandra (6 marks), virtual nodes (5 marks)

2020: Quorum-based strong consistency condition (6 marks)

2019: Consistency level for read-your-own-write with min write latency (7 marks)

2018: Cassandra consistency levels for single-datacenter (8 marks)

5.1 What is Cassandra?

Distributed peer-to-peer key-value store. Combines Google's BigTable data model with Amazon's Dynamo distribution. No single point of failure (no master). Released by Facebook (2008), Apache project (2010). Focused on Availability and Partition Tolerance (AP in CAP). Follows BASE: Basically Available, Soft State, Eventually Consistent.

Architecture

Cassandra is a distributed hash table. Machines called Nodes, arranged in a logical ring. Each node contains: Commit Log (WAL), MemTables, SSTables, Index, and a Coordinator for inter-node communication. Uses Gossip Protocol: every second, each node sends a token with its info + info about known nodes to 3 other nodes. Eventually all nodes have global knowledge.

Partitioning

RandomPartitioner: Hash function produces random output. Simple but no load balancing guarantees. Good for point queries.

OrderPreservingPartitioner: If $x \leq y$ then $\text{hash}(x) \leq \text{hash}(y)$. Enables range queries but can cause data clustering (hot spots).

Cassandra combines both: order-preserving hash to maintain key ordering, then consistent hashing to assign to nodes.

Replication and Arcs

Data assigned to nodes based on partition key hash. If no exact match, select next node clockwise in ring. With replication factor N , data is copied to the next N nodes clockwise. These N nodes form an 'arc' responsible for that partition key. Nodes within an arc monitor each other; on failure, the arc extends to the next available node.

Virtual Nodes (asked 2023)

Problem: Hash-based assignment doesn't spread data evenly, and machines have different capacities. Solution: Each physical node represented as multiple virtual nodes spread around the ring. A machine with 8 CPU cores gets more virtual nodes than one with 4 cores. This balances load across machines with different capabilities.

Consistency Levels (asked 2018, 2019, 2020)

Level	Write Behavior	Read Behavior
ANY	At least 1 node (including hinted handoff)	Not supported
ONE	1 replica commits	Returns first replica to respond
QUORUM	$N/2+1$ replicas commit	Most recent timestamp from $N/2+1$ replies
LOCAL_QUORUM	$N/2+1$ within local datacenter	Same, within local DC
ALL	All N replicas commit	Most recent after all respond

Quorum Formula (asked repeatedly)

KEY FORMULA: Strong Consistency via Quorum

For strong consistency (linearizability): $R + W > N$ AND $2W > N$

Where N = replication factor, R = read contacts, W = write contacts.

This guarantees every read sees the latest write (at least one node overlaps).

Example (2019): Replication factor 5, want read-your-own-write with minimum write latency. Answer: QUORUM/QUORUM. $R=3$, $W=3$, $N=5$. $R+W=6>5$. Both ANY/ONE and ONE/ONE fail because single-copy writes cannot guarantee the read replica has the latest data.

Cassandra vs HBase

Feature	HBase	Cassandra
Architecture	Master-worker (HMaster)	Peer-to-peer (no master)
CAP focus	Consistency + Availability	Availability + Partition Tolerance
Single point of failure	Yes (HMaster)	No
Multi-datacenter	Complex	Native support
Consistency	Strong	Tunable (eventual to strong)
Storage engine	LSM tree on HDFS	LSM tree (local)
Data model	<RowKey, Column, Timestamp>	<Key, Column> (no default timestamp)

6. Hadoop (8 appearances, 49 marks)

EXAM PATTERN: Appeared in 5 years. Focus on architecture, overheads, YARN.

2025: Hardware/config design decisions for deployment (6 marks), batch limitations (4 marks)

2023: Profile overhead costs (6-8 marks), YARN vs traditional Hadoop (3 marks), YARN containers vs modern containers (8 marks)

2018: NameNode recovery (10 marks)

6.1 Hadoop Architecture (v1)

Client: Submits work to cluster, communicates with Job Tracker.

Job Tracker: Receives work, uses NameNode to find data locations, assigns tasks to Task Trackers. Handles fault tolerance, speculative execution.

Task Tracker: Executes Map and Reduce tasks. Has multiple fixed slots. Reports via heartbeats.

23-Step Job Execution Flow

1. Client calls Run Job → 2. Gets Job ID from JobTracker → 3. Copies Job.jar to HDFS → 4. Submits job → 5. JobTracker initializes → 6. Retrieves Input Splits from NameNode → 7. Registers tasks with TaskTrackers → 8. TaskTracker gets Job.jar from HDFS → 9. Launches JVM → 10. Map executes → 11-13. Sort + write intermediate output → 14. Map complete reported → 15-18. Reduce tasks assigned and launched → 19-21. Reduce executes and writes → 22-23. Completion reported.

Overhead Costs (asked 2023)

Slow data loading (small files problem with HDFS), intermediate sort cost, network transfer latencies during shuffle phase, disk write costs after map sort, HDFS read/write overheads, job startup/initialization costs.

Data Locality Principle

Move compute to the data, not data to compute. JobTracker assigns tasks based on: Data-local (best) > Rack-local > Remote (worst). Map tasks check where input splits are stored. Reduce tasks check where items for each key reside.

YARN (Yet Another Resource Negotiator)

Separates cluster management from job coordination. Resource Manager manages the cluster (Scheduler + Application Manager). Node Manager replaces Task Tracker with flexible resource containers. Application Master replaces Job Tracker for individual jobs, runs on worker nodes.

Key improvement: Job management delegated to Application Master on worker nodes, reducing central bottleneck. Supports more than just MapReduce jobs.

YARN Containers vs Modern Containers (asked 2023 M-level)

YARN containers are resource allocation units (CPU + memory blocks) managed by Node Manager. Modern containers (Docker) are software sandboxes with isolated filesystems and dependencies. YARN containers are lightweight resource partitions; Docker containers provide full software isolation.

Hadoop Versions

V1: Basic MapReduce + HDFS. V2: Adds YARN for better scheduling. V3: Multiple NameNode support, erasure coding (reduced storage overhead), container and GPU support.

7. Big Data Fundamentals (7 appearances, 39 marks)

EXAM PATTERN: Typically Question 1 in recent exams. Expect 6-15 marks.

Common formats: Identify Big Data dimensions for a scenario, justify why something is/isn't Big Data, explain vertical vs horizontal scaling challenges.

2025: Is tracking a Big Data problem? (3 marks), why video processing IS Big Data (6 marks)

2023: Satellite imaging Big Data dimensions (6 marks), why mainframe is insufficient (8 marks), horizontal scaling challenges (8 marks)

2019: CAP theorem — which system type for chat vs banking (8 marks)

7.1 The 6 Dimensions (V's)

Volume: Sheer size of data. Quantify it! (e.g., '500MB/sec from 100 satellites').

Velocity: Speed at which data arrives and must be processed. Link to real-time requirements.

Variety: Different data types — structured, semi-structured, unstructured (text, video, audio, images).

Veracity: Data quality, accuracy, trustworthiness. Sensor errors, noise, missing values.

Variability: Data flow varies over time — peaks and valleys tied to real-world events.

Complexity: Entities, hierarchies, relationships between data items.

EXAM TECHNIQUE: Identifying Big Data Dimensions

1. Pick 2-3 dimensions most relevant to the scenario (not all 6).
2. For EACH dimension: name it (1 mark), explain WHY it applies with specific numbers/facts from the scenario (1-2 marks).
3. Not everything is Big Data! If the scenario describes small, simple data, say so (see 2025 Q1a: tracking goods was NOT Big Data).

Vertical vs Horizontal Scaling

Vertical: Buy bigger machines (mainframes). Limited by reticle limit (physical chip size), transistor shrinking costs, single point of failure, cost-ineffective.

Horizontal: Distribute work across many cheap machines. Infinitely scalable in theory. Challenges: load balancing, communication overheads, fault tolerance, orchestration, programming complexity.

CAP Theorem (asked 2019)

A distributed system can guarantee only 2 of 3 properties: Consistency (all nodes see same data), Availability (every request gets a response), Partition Tolerance (system works despite network failures).

Combination	Properties	Examples
CA	Consistency + Availability	Traditional RDBMS (Oracle, MySQL)
CP	Consistency + Partition Tolerance	MongoDB, HBase
AP	Availability + Partition Tolerance	Cassandra, CouchDB

BASE vs ACID

ACID (traditional databases): Atomicity, Consistency, Isolation, Durability — strong guarantees but limited scalability.

BASE (Big Data systems): Basically Available, Soft State, Eventually Consistent — weaker consistency guarantees but higher availability and scalability.

Batch vs Stream Processing

Batch: Large blocks analyzed at once, concurrent processing, then merge. Cares about completion time. Example: daily report generation.

Stream: Items processed individually in a pipeline. Cares about per-item latency. Example: real-time fraud detection.

8. Data Structures: LSM Trees & Bloom Filters (4 appearances, 28 marks)

EXAM PATTERN: Growing importance. Appeared 2018, 2020, 2025.

2025: HBase RegionServer storage mechanism (LSM tree) for fast access (7 marks), Bloom filter $O(k)$ complexity (3 marks)

2020: True/false on LSM tree properties (8 marks)

2018: High write throughput mechanisms in NoSQL stores (10 marks)

8.1 LSM Trees (Log-Structured Merge Trees)

The storage engine used by both HBase and Cassandra. Designed for high write throughput with acceptable read performance.

Components

Write-Ahead Log (WAL): Append-only persistent log. Every write goes here first. Sequential writes = fast. Provides crash recovery.

MemTable: In-memory store of recent writes, keyed to files. Checked first on reads (acts as cache). When full, sorted and flushed to disk as an SSTable.

SSTables (Sorted String Tables): Immutable on-disk files created from MemTable flushes. Sorted by key for efficient scanning.

Index: In-memory mapping between keys and which SSTables contain them. Uses Bloom Filters for $O(1)$ lookups.

Compactor: Background process that merges SSTables to reduce fragmentation, applies tombstones (deletes), reclaims space.

Write Process

1. Append record to WAL (fast sequential write). 2. Add record to MemTable. 3. When MemTable is full: sort it, write as new SSTable (spill file), update Index, clear WAL. 4. Compactor periodically merges SSTables in background.

Read Process

1. Check MemTable (most recent data, fastest). 2. Query Index using Bloom Filter to find candidate SSTables ($O(1)$). 3. Load and scan relevant SSTables (sequential block reads = fast).

Trade-offs (asked 2018, 2020)

Writes are fast: WAL is append-only, MemTable edits are in-memory, disk writes are batched. Reads of recent data are fast via MemTable cache. Reads of older data may require scanning multiple SSTables (slower). Updates and deletes are expensive — require compaction. Compaction is resource-intensive but can be scheduled during quiet periods.

8.2 Bloom Filters

Space-efficient probabilistic data structure. Returns 'possibly in set' or 'definitely NOT in set'.
Used in LSM trees to quickly determine which SSTables to read.

How They Work

1. Define a bit array of m bits, all initialized to 0. 2. Define k hash functions. 3. To ADD an item: hash it with all k functions, set those bit positions to 1. 4. To QUERY: hash the query with all k functions. If ALL positions are 1 = possibly present. If ANY position is 0 = definitely not present.

Key Formulas

BLOOM FILTER FORMULAS

False positive probability: $p = (1 - e^{(-kn/m)})^k$

Optimal k : $k = (m/n) \times \ln(2)$

Optimal m : $m = -(n \times \ln(p)) / (\ln(2))^2$

Where m = bit array size, k = number of hash functions, n = number of items, p = target false positive rate.

Example: $n=1000$ items, target 5% false positive rate → compute m and k from formulas.

9. Cluster Management & Containers (2 appearances, 13 marks)

EXAM PATTERN: New topic appearing in 2023-2025. Expect growth.

2025: What is Kubernetes and how it helps with deployment/failures (5 marks)

2023 M-level: YARN containers vs modern containers (8 marks)

9.1 Software Sandboxes

JVM: Solves application and library level incompatibilities. Multiple JVMs can run simultaneously. Used by Hadoop and Spark.

Virtual Machines: Full OS isolation. Solves ALL incompatibilities. But: resource-heavy, large images, hypervisor overhead. Not great for Big Data.

Containers (Docker): Lightweight OS-level isolation. Shares host OS kernel. Solves app, library, and executable incompatibilities but NOT driver incompatibilities. Each container runs a single command, destroyed on exit. Changes lost — output must be written to mounted storage.

9.2 Kubernetes

Open-source container orchestration system (Google, now CNCF). Automates deployment, scaling, and management of containerized applications.

Components

Master: API server, ETCD (central database), Scheduler (allocates containers), Controller Manager (maintenance, recovery).

Worker: Kubelet (ensures pods are running), Kube-proxy (network rules), cAdvisor (resource monitoring).

Building Blocks

Pods: Smallest compute unit. 1+ containers that share resources, same machine, visible on network, created/destroyed together.

Services: Stable hostname for reaching Pods (whose location may change). Load balancing across multiple Pods.

Routes: Expose services externally at a defined hostname.

Kubernetes works as a state management system: users define desired state, Kubernetes continuously works to achieve it, handling failures automatically.

10. Legal & Ethical Considerations (1 appearance, 6 marks)

EXAM PATTERN: New in 2025. Expect to be mixed into other questions.

2025: Legal/ethical implications of training AI from collected video data (6 marks)

Week 10 content gets mixed into Questions 1-3 per examiner's guide.

Four Core Ethical Concepts

- 1. Privacy:** Human right (ECHR, DPA 1998). Difficult to distinguish public vs private in social media. Even 'public' data has privacy implications.
- 2. Informed Consent:** Individual must consent to clearly understood uses. In Big Data, future uses are often unknown at collection time. T&C-based consent is debatable.
- 3. De-identification:** Removing identifying elements. Now seen as a continuum, not binary. Risks increase with more variables and linkable datasets. Risks can be mitigated but NOT eliminated.
- 4. Research Integrity:** Clear provenance of data sources and processing. Data repositories support validation and replication.

Key Laws

GDPR: Protects personal data of identifiable individuals. Requires: explicit consent, declared lawful basis and purpose, stated retention period, Data Protection Officer. Processing only if: consent, contract, legal obligation, vital interests, public interest, or legitimate interests.

DPA 1998: 8 principles: fair/lawful processing, specified purposes, adequate/relevant/not excessive, accurate, not kept too long, respect data subject rights, appropriate security, no transfer outside EEA without protection.

11. Batch vs Streaming (3 appearances, 12 marks)

EXAM PATTERN: Growing topic. Appeared 2023-2025.

2025: Why computing every 10 seconds is difficult in Hadoop (4 marks), why streaming improves resource usage for ML (4 marks)

2023 Practice: Batch vs stream for video encoding (4 marks)

Batch: large blocks analyzed concurrently, cares about completion time. Stream: items processed in pipeline, cares about per-item latency.

Why Streaming is Challenging

No discrete equal-sized blocks (load balancing is harder), velocity is variable, end-to-end latency is critical (human user waiting), buffering between stages causes back pressure, stateful processing requires routing specific items to specific instances.

Back Pressure

If items arrive faster than processing, input buffers fill up. Full buffer stops accepting new items, preventing previous stage from emptying its output buffer. This cascades backward through the entire pipeline. Once all buffers are full, the job usually fails.

Why Hadoop is Bad for Streaming (asked 2025)

Hadoop requires key-based operations (Map/Reduce both need keys). Very regular computation (e.g., every 10 seconds) is inefficient because Hadoop has significant job startup costs each time. No native support for stateful processing.

12. ZooKeeper & DHT / Overlay Networks

12.1 ZooKeeper (1 appearance, 3 marks)

Distributed key-value store with change notifications. Hierarchical namespace of ZNodes. Replicated across 5+ servers (Leader + Followers). Used by HBase, Spark, Flink. Solves: how to track state of distributed component replicas without relying on the application itself.

Use cases: file locking, global configuration, access controls, bootstrap configuration, service replica discovery.

12.2 DHT / Overlay Networks (2 appearances, 14 marks)

EXAM PATTERN: Asked in 2020 and 2023.

2023: Consistent hashing + order-preserving hashing in Cassandra (6 marks)

2020: True/false on consistent hashing properties (8 marks)

Consistent Hashing

Peers assigned IDs in same hash space as file keys. Key X assigned to first peer with ID $\geq X$ (clockwise in ring). When peer joins: requests index data for its ID range. When peer leaves: passes data to next peer. Enables masterless index distribution.

Finger Tables

Optimization: instead of $O(n)$ hops around ring, each node stores $\log_2(N)$ entries pointing to specific nodes. Lookup takes $O(\log N)$ hops. Each node needs only $\log_2(N)$ entries — no global knowledge required.

Order-Preserving Hashing

If $x \leq y$ then $\text{hash}(x) \leq \text{hash}(y)$. Enables range queries. Combined with consistent hashing in Cassandra for data placement.

13. Exam Strategy & Top Expected Questions

13.1 Exam Format (2026)

60 marks, 1.5 hours, 3 compulsory questions, closed book (crib sheet with Spark functions provided), Moodle Quiz format.

Question 1: Theory from weeks 1-2. Question 2: Distributed compute. Question 3: Distributed data. Week 10 content mixed in.

13.2 Answer Technique

Discuss/Describe questions: marks ÷ 2 = approximate number of sentences needed.

List questions: look for 'X points are expected' in guidance.

Context matters: few marks for pure bookwork. Link every answer to the specific scenario given.

13.3 Top 10 Most Expected Questions for 2026

PREDICTION: Most Likely Exam Questions

1. Identify Big Data dimensions for a given scenario (6 marks) — appeared 2023, 2025
2. Design a MapReduce job for a new scenario (9-18 marks) — appeared every year
3. Spark advantages over Hadoop (6-8 marks) — appeared 2023, 2025
4. HDFS read/write process or NameNode recovery (8-10 marks) — appeared 2015-2020
5. Cassandra consistency levels / quorum calculation (6-8 marks) — appeared 2018-2020
6. LSM tree write/read process in HBase (7-10 marks) — appeared 2018, 2020, 2025
7. Horizontal scaling challenges for a scenario (8 marks) — appeared 2023
8. Batch vs streaming trade-offs (4-6 marks) — appeared 2023, 2025
9. Legal/ethical implications of data collection (6 marks) — appeared 2025
10. Kubernetes or cluster management role (5-8 marks) — appeared 2023, 2025

13.4 Key Formulas to Memorize

Speed-up: $\text{Speed-up}(n,p) = \text{Serial_Time}(n) / \text{Parallel_Time}(n,p)$

Quorum: $R + W > N$ AND $2W > N$ for strong consistency

Bloom Filter FP rate: $p = (1 - e^{(-kn/m)})^k$

Optimal Bloom k: $k = (m/n) \times \ln(2)$

Optimal Bloom m: $m = -(n \times \ln(p)) / (\ln(2))^2$

Finger table size: $\log_2(N)$ entries per node

DHT lookup: $O(\log N)$ hops with finger tables